From Neurons to Transformers

A coherent path from linear models to attention mechanisms

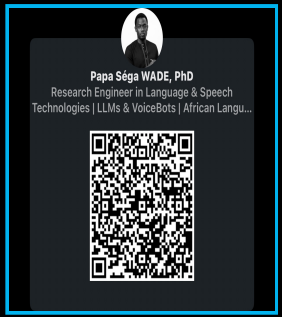
Dr. Papa-Séga WADE

AIMS Senegal | Session 1 <> Applied GAAI

MODULE 0 - 4-hour module support aligned with the official syllabus

"Every equation you learn is a tool to build the AI of tomorrow"

Who am I?



papasegawade.com

Dr. Papa-Séga WADE

- Senegal → **X-Telecom Paris** → PhD IMT Atlantique
- Agentic & Speech AI Expert, **Orange Innovation**
- Creator of the **first Wolof voicebot** in production
- Demos at **MWC Barcelona 2026**, AfricaTech, GITEX
- **Founder of MathsPSW-IA**

Mission

Transforming complex algorithms into concrete solutions for the African continent.

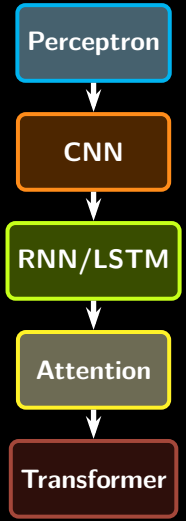
papa-sega.wade@m4x.org /LinkedIn/papa sega wade

Why This Module Matters

Course Goal

Give all students, regardless of background, the keys to understand why the Transformer exists and what problem it solves.

- **We are not repeating a full deep learning course.**
- We build one clear narrative: **linear models → neural networks → sequence models → attention → Transformers**.
- We start from intuition and visualization before formal notation.
- Advanced math is available as an optional layer, not a prerequisite.

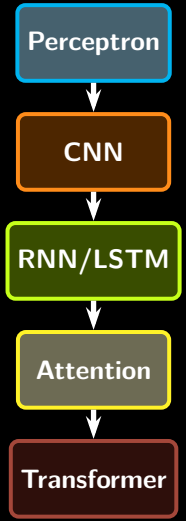


Why This Module Matters

Course Goal

Give all students, regardless of background, the keys to understand why the Transformer exists and what problem it solves.

- **We are not repeating a full deep learning course.**
- We build one clear narrative: **linear models** → **neural networks** → **sequence models** → **attention** → **Transformers**.
- We start from intuition and visualization before formal notation.
- Advanced math is available as an optional layer, not a prerequisite.

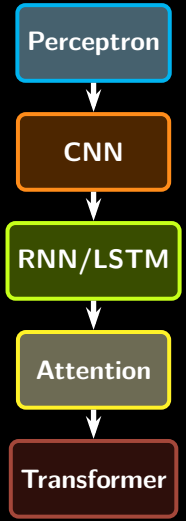


Why This Module Matters

Course Goal

Give all students, regardless of background, the keys to understand why the Transformer exists and what problem it solves.

- **We are not repeating a full deep learning course.**
- We build one clear narrative: **linear models** → **neural networks** → **sequence models** → **attention** → **Transformers**.
- We start from intuition and visualization before formal notation.
- Advanced math is available as an optional layer, not a prerequisite.

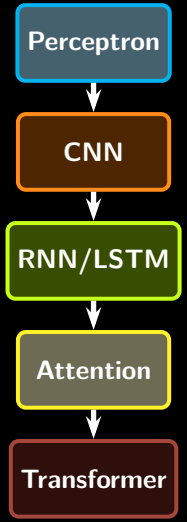


Why This Module Matters

Course Goal

Give all students, regardless of background, the keys to understand why the Transformer exists and what problem it solves.

- **We are not repeating a full deep learning course.**
- We build one clear narrative: **linear models** → **neural networks** → **sequence models** → **attention** → **Transformers**.
- We start from intuition and visualization before formal notation.
- Advanced math is available as an optional layer, not a prerequisite.



What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)
- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)
- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)
- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)
- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)
- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

∼40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)
- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)
- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)
- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)
- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)
- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)
- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)
- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)
- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)
- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)
- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)
- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)
- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)
- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)
- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)
- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)
- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)
- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)
- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)
- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)
- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)
- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)
- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)
- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)
- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)
- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)

- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)

- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)

- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)

- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)

- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)

- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)

- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)

- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)

- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)

- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)

- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)

- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)

- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)

- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)

- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)

- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)

- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)

- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)

- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)

- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)

- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)

- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)

- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)

- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)

- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)

- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)

- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)

- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)

- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)

- S12: Security & bias
- S13–S15: Final project

We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

What We Will Cover Together (30h)

M0: Neurons → Transformers (4h)

- S1: From Perceptron to Attention
- S2: The Transformer architecture

M1: Understanding LLMs (4h)

- S3: GenAI panorama & LLM pipeline
- S4: Open models, costs & Speech

M2: Using & Adapting LLMs (8h)

- S5: Prompt engineering
- S6: RAG from scratch
- S7: Fine-tuning (LoRA/QLoRA)
- S8: Evaluation & mid-project

M3: Agents & Autonomous Systems (6h)

- S9: Intro to LLM Agents
- S10: Memory & workflows
- S11: Multi-agent project

M4: Security, Eval & Project (8h)

- S12: Security & bias
- S13–S15: Final project

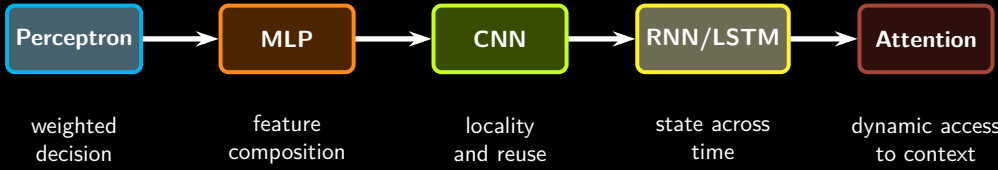
We assume basic Python, vectors, matrices, and optimization. We do **not** assume prior exposure to neural architectures or LLM internals.

~40% hands-on labs per session

Module Roadmap (4h)

1. Session 1: From Neurons to Attention
2. Session 2: The Transformer
3. Wrap-Up

The Story of Architectural Progress



Key idea

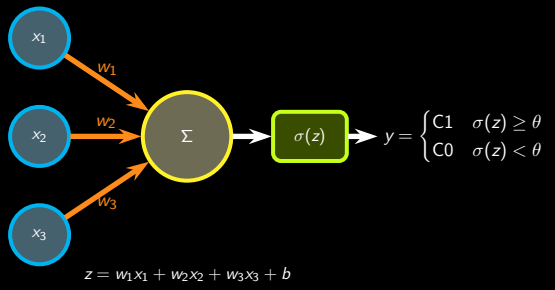
Every new architecture adds an **inductive bias**: a useful built-in assumption about the structure of data.

Perceptron: A Weighted Decision Rule

Moderate analogy

A perceptron behaves like a **committee vote**: each signal has a weight, all weighted signals are summed, then a decision is made.

- Inputs: evidence or features
- Weights: importance of each feature
- Bias: decision offset
- Activation: turn score into a usable output

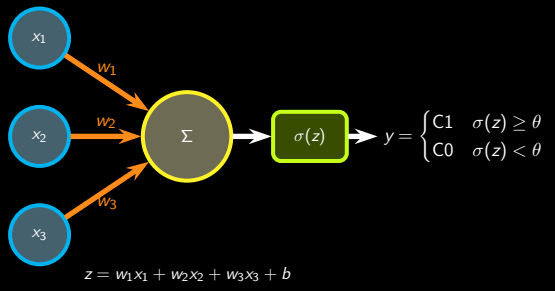


Perceptron: A Weighted Decision Rule

Moderate analogy

A perceptron behaves like a **committee vote**: each signal has a weight, all weighted signals are summed, then a decision is made.

- Inputs: evidence or features
- Weights: importance of each feature
- Bias: decision offset
- Activation: turn score into a usable output

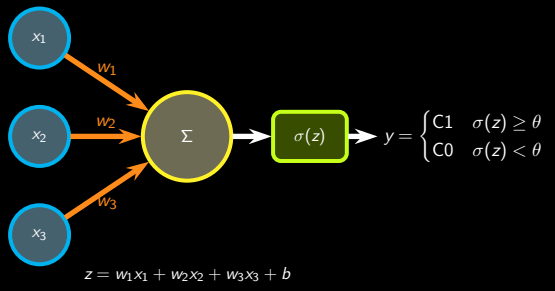


Perceptron: A Weighted Decision Rule

Moderate analogy

A perceptron behaves like a **committee vote**: each signal has a weight, all weighted signals are summed, then a decision is made.

- Inputs: evidence or features
- Weights: importance of each feature
- Bias: decision offset
- Activation: turn score into a usable output

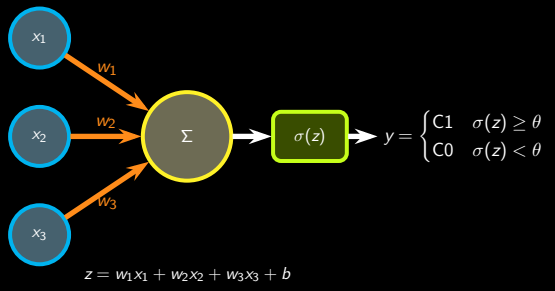


Perceptron: A Weighted Decision Rule

Moderate analogy

A perceptron behaves like a **committee vote**: each signal has a weight, all weighted signals are summed, then a decision is made.

- Inputs: evidence or features
- Weights: importance of each feature
- Bias: decision offset
- Activation: turn score into a usable output

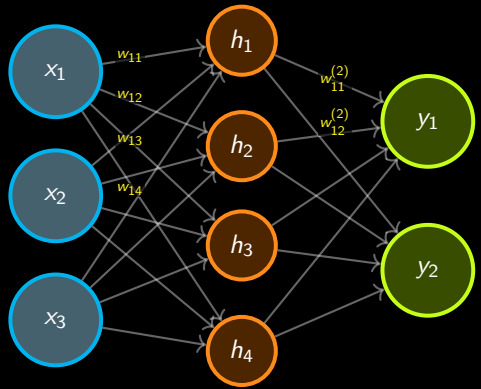


From Perceptron to MLP

Why one neuron is not enough

A single perceptron draws a **linear decision boundary**. Many real problems need several layers of transformations.

- Layer 1 learns simple combinations
- Layer 2 recombines them into richer features
- Non-linearity lets the network bend the decision boundary



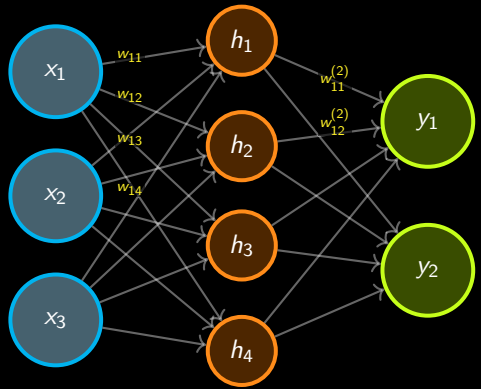
$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \quad \mathbf{y} = \sigma(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$

From Perceptron to MLP

Why one neuron is not enough

A single perceptron draws a **linear decision boundary**. Many real problems need several layers of transformations.

- Layer 1 learns simple combinations
- Layer 2 recombines them into richer features
- Non-linearity lets the network bend the decision boundary



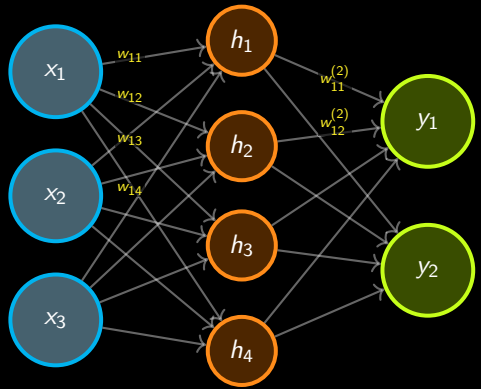
$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \quad \mathbf{y} = \sigma(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$

From Perceptron to MLP

Why one neuron is not enough

A single perceptron draws a **linear decision boundary**. Many real problems need several layers of transformations.

- Layer 1 learns simple combinations
- Layer 2 recombines them into richer features
- Non-linearity lets the network bend the decision boundary



$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \quad \mathbf{y} = \sigma(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$

How the computer "sees" an image?

Revelation: The computer does NOT see like we do!

What YOU see:

What the COMPUTER sees:

255	203	178	230	152
120	89	50	165	102
0	217	128	76	0
38	0	242	0	64
140	191	0	115	89

A matrix of numbers!

Each pixel: **0** (black) to **255** (white).

"It's a face!"

You instantly recognize a face, identity.

The Challenge

Transform this matrix of numbers into pattern recognition. **Solution:** the **CNN**, with an inductive bias of **locality**.

How the computer "sees" an image?

Revelation: The computer does NOT see like we do!

What YOU see:

What the COMPUTER sees:

255	203	178	230	152
120	89	50	165	102
0	217	128	76	0
38	0	242	0	64
140	191	0	115	89

A matrix of numbers!

Each pixel: **0** (black) to **255** (white).

"It's a face!"

You instantly recognize a face, identity.

The Challenge

Transform this matrix of numbers into pattern recognition. **Solution:** the **CNN**, with an inductive bias of **locality**.

How the computer "sees" an image?

Revelation: The computer does NOT see like we do!

What YOU see:

What the COMPUTER sees:

255	203	178	230	152
120	89	50	165	102
0	217	128	76	0
38	0	242	0	64
140	191	0	115	89

A matrix of numbers!

Each pixel: **0** (black) to **255** (white).

"It's a face!"

You instantly recognize a face, identity.

The Challenge

Transform this matrix of numbers into pattern recognition. **Solution:** the **CNN**, with an inductive bias of **locality**.

How the computer "sees" an image?

Revelation: The computer does NOT see like we do!

What YOU see:

What the COMPUTER sees:

255	203	178	230	152
120	89	50	165	102
0	217	128	76	0
38	0	242	0	64
140	191	0	115	89

A matrix of numbers!
Each pixel: **0** (black) to **255** (white).

"It's a face!"
You instantly recognize a face, identity.

The Challenge

Transform this matrix of numbers into pattern recognition. **Solution:** the **CNN**, with an inductive bias of **locality**.

How the computer "sees" an image?

Revelation: The computer does NOT see like we do!

What YOU see:

What the COMPUTER sees:

255	203	178	230	152
120	89	50	165	102
0	217	128	76	0
38	0	242	0	64
140	191	0	115	89

A matrix of numbers!
Each pixel: **0** (black) to **255** (white).

"It's a face!"

You instantly recognize a face, identity.

The Challenge

Transform this matrix of numbers into pattern recognition. **Solution:** the **CNN**, with an inductive bias of **locality**.

How the computer "sees" an image?

Revelation: The computer does NOT see like we do!

What YOU see:

What the COMPUTER sees:

255	203	178	230	152
120	89	50	165	102
0	217	128	76	0
38	0	242	0	64
140	191	0	115	89

A matrix of numbers!

Each pixel: **0** (black) to **255** (white).

"It's a face!"

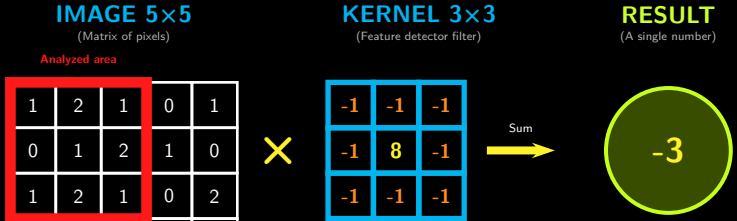
You instantly recognize a face, identity.

The Challenge

Transform this **matrix of numbers** into pattern recognition. **Solution:** the **CNN**, with an inductive bias of **locality**.

CNN: The Convolution Mechanism (1/2)

How does the CNN detect patterns in the image?



Detailed Calculation:

$(-1 \times 1) + (-1 \times 2) + (-1 \times 1)$ (row 1)
$+(-1 \times 0) + (8 \times 1) + (-1 \times 2)$ (row 2)
$+(-1 \times 1) + (-1 \times 2) + (-1 \times 1)$ (row 3)
$= -1 - 2 - 1 + 0 + 8 - 2 - 1 - 2 - 1 = \mathbf{-3}$

The Convolution Mechanism (2/2)

1. The Formula: A Double Sum

$$(I * K)[i,j] = \sum_{m=0}^{2} \sum_{n=0}^{2} \underbrace{I[i+m, j+n]}_{\text{Image Pixels}} \times \underbrace{K[m,n]}_{\text{Filter Weights}}$$

Don't be afraid: it's just multiplication followed by addition!

2. The Meaning: Contrast Detector

High Result ⟶ **Edge Detected** (Sudden change)

Low Result ⟶ **Uniform Area** (Nothing to report)

3. The Process: The "Scan"

The CNN moves this small window (the Kernel) over **the entire image**, pixel by pixel, to build an Activation Map. This represents the inductive bias of **locality and reuse**.

Exercise: Convolution Step-by-Step

How the CNN Detects Vertical Edges

Image 3×3:

$$I = \begin{bmatrix} 1 & 0 & 2 \\ 0 & \mathbf{1} & 1 \\ 2 & 0 & 1 \end{bmatrix}$$

Kernel 3×3:

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Question: Calculate the convolution at the yellow pixel

Reminder of the formula:

$$(I * K) = \sum_{i,j} I[i,j] \times K[i,j]$$

1. Element-wise product
2. Sum of all products

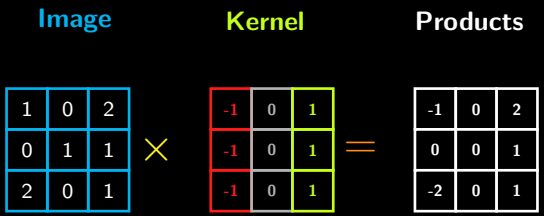
Image with central pixel

1	0	2
0	1	1
2	0	1

Center

Exercise: Complete Resolution

Step 1: Element-wise Products



Step 2: Sum of All Products

$$\text{Result} = (-1) + (0) + (2) + (0) + (0) + (1) + (-2) + (0) + (1)$$
$$= \mathbf{1}$$

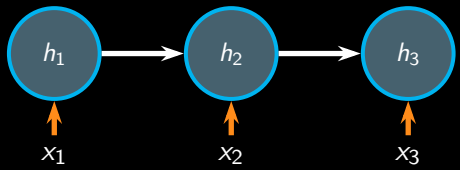
Result = 1 → Presence of a vertical edge

RNNs: When Order Matters

Why RNNs appeared

Images are mostly spatial. Language is **sequential**. The order of words changes meaning.

- The model reads one token at a time
- Hidden state h_t carries context forward
- The same transition is reused at every time step

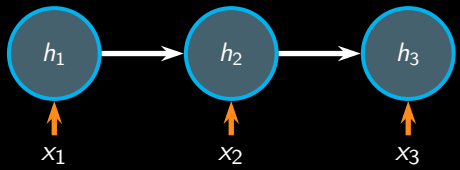


RNNs: When Order Matters

Why RNNs appeared

Images are mostly spatial. Language is **sequential**. The order of words changes meaning.

- The model reads one token at a time
- Hidden state h_t carries context forward
- The same transition is reused at every time step

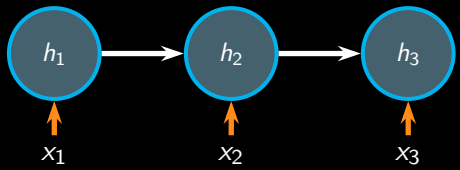


RNNs: When Order Matters

Why RNNs appeared

Images are mostly spatial. Language is **sequential**. The order of words changes meaning.

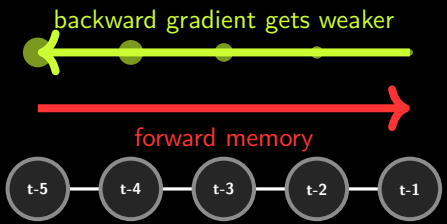
- The model reads one token at a time
- Hidden state h_t carries context forward
- The same transition is reused at every time step



The Sequence Problem and Vanishing Gradients

In long sequences, the signal from early tokens becomes hard to preserve and hard to learn through backpropagation.

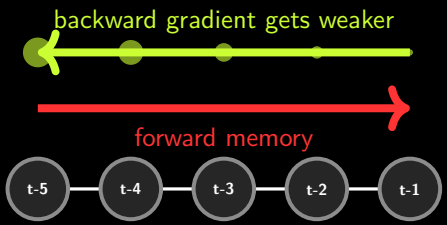
- Recent words dominate
- Early information fades
- Training becomes unstable on long-range dependencies



The Sequence Problem and Vanishing Gradients

In long sequences, the signal from early tokens becomes hard to preserve and hard to learn through backpropagation.

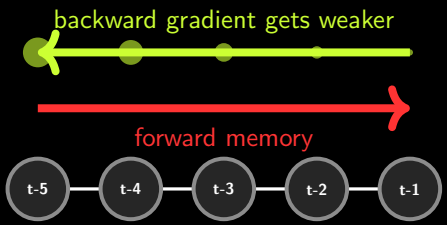
- Recent words dominate
- Early information fades
- Training becomes unstable on long-range dependencies



The Sequence Problem and Vanishing Gradients

In long sequences, the signal from early tokens becomes hard to preserve and hard to learn through backpropagation.

- Recent words dominate
- Early information fades
- Training becomes unstable on long-range dependencies



LSTM: A Better Memory Mechanism

Useful analogy: a notebook you can selectively update

Forget gate

Remove information that no longer helps.

Input gate

Write new information into memory.

Output gate

Expose the part needed now.

Cell state = long-term memory notebook

erase **write** **read**

Why it helped

LSTMs preserve useful information better than plain RNNs, but still process sequences step by step.

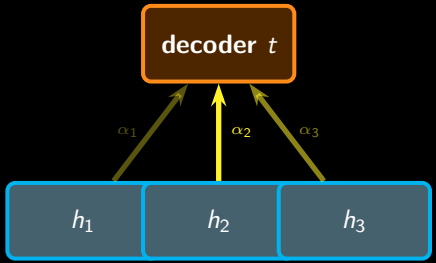
Bahdanau Attention: Stop Compressing Everything

The bottleneck

Encoder-decoder RNNs often tried to compress the whole input sentence into one fixed vector.

The idea of attention

Instead of forcing the decoder to remember everything, let it **look back** at the relevant encoder states at each step.



- Attention scores tell us what to focus on now
- Context becomes dynamic, not frozen
- This is the bridge to Transformers

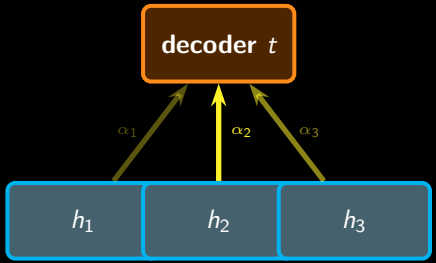
Bahdanau Attention: Stop Compressing Everything

The bottleneck

Encoder-decoder RNNs often tried to compress the whole input sentence into one fixed vector.

The idea of attention

Instead of forcing the decoder to remember everything, let it **look back** at the relevant encoder states at each step.



- Attention scores tell us what to focus on now
- Context becomes dynamic, not frozen
- This is the bridge to Transformers

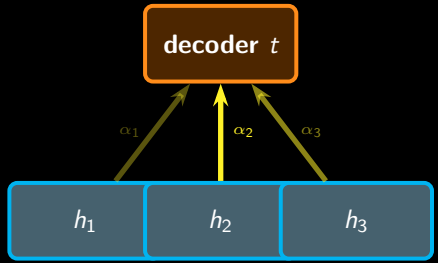
Bahdanau Attention: Stop Compressing Everything

The bottleneck

Encoder-decoder RNNs often tried to compress the whole input sentence into one fixed vector.

The idea of attention

Instead of forcing the decoder to remember everything, let it **look back** at the relevant encoder states at each step.



- Attention scores tell us what to focus on now
- Context becomes dynamic, not frozen
- This is the bridge to Transformers

Git and GitHub Basics for Pair Work

Minimal workflow

- **clone**: get the project
- **add/commit**: save work locally
- **push**: publish your changes
- **pull**: sync with your teammate

```
git clone <repo-url>
# edit notebook or script
git add .
git commit -m "feat: add attention demo"
git push origin main
git pull origin main
```

Git and GitHub Basics for Pair Work

Minimal workflow

- **clone**: get the project
- **add/commit**: save work locally
- **push**: publish your changes
- **pull**: sync with your teammate

```
git clone <repo-url>
# edit notebook or script
git add .
git commit -m "feat: add attention demo"
git push origin main
git pull origin main
```

Git and GitHub Basics for Pair Work

Minimal workflow

- **clone**: get the project
- **add/commit**: save work locally
- **push**: publish your changes
- **pull**: sync with your teammate

```
git clone <repo-url>
# edit notebook or script
git add .
git commit -m "feat: add attention demo"
git push origin main
git pull origin main
```

Git and GitHub Basics for Pair Work

Minimal workflow

- **clone**: get the project
- **add/commit**: save work locally
- **push**: publish your changes
- **pull**: sync with your teammate

```
git clone <repo-url>
# edit notebook or script
git add .
git commit -m "feat: add attention demo"
git push origin main
git pull origin main
```

Session 1 Lab: From Char-LSTM to Attention

Baseline (30 min) — `01_baseline.py`

Train a 2-layer character-level LSTM on a **Wolof** corpus.
Generate text from 3 prompts (short / medium / long) and annotate where the model drifts.

What to annotate

- At which offset (∼chars) does drift start?
- What repetition patterns appear?
- Does the model forget the initial context?
- Does a longer prompt improve coherence?

Extension (+20 min) — `02_extension.py`

Build a **TinyAttnLM**: same data, same training loop, but replace the LSTM with a single self-attention layer + causal mask. Compare qualitatively with the LSTM outputs.

`labs/lab_s1/`

Session 1 Lab: From Char-LSTM to Attention

Baseline (30 min) — `01_baseline.py`

Train a 2-layer character-level LSTM on a **Wolof** corpus.
Generate text from 3 prompts (short / medium / long) and annotate where the model drifts.

What to annotate

- At which offset (∼chars) does drift start?
- What repetition patterns appear?
- Does the model forget the initial context?
- Does a longer prompt improve coherence?

Extension (+20 min) — `02_extension.py`

Build a **TinyAttnLM**: same data, same training loop, but replace the LSTM with a single self-attention layer + causal mask. Compare qualitatively with the LSTM outputs.

`labs/lab_s1/`

Session 1 Lab: From Char-LSTM to Attention

Baseline (30 min) — `01_baseline.py`

Train a 2-layer character-level LSTM on a **Wolof** corpus.
Generate text from 3 prompts (short / medium / long) and annotate where the model drifts.

What to annotate

- At which offset (∼chars) does drift start?
- What repetition patterns appear?
- Does the model forget the initial context?
- Does a longer prompt improve coherence?

Extension (+20 min) — `02_extension.py`

Build a **TinyAttnLM**: same data, same training loop, but replace the LSTM with a single self-attention layer + causal mask. Compare qualitatively with the LSTM outputs.

`labs/lab_s1/`

Session 1 Lab: From Char-LSTM to Attention

Baseline (30 min) — `01_baseline.py`

Train a 2-layer character-level LSTM on a **Wolof** corpus.
Generate text from 3 prompts (short / medium / long) and annotate where the model drifts.

What to annotate

- At which offset (∼chars) does drift start?
- What repetition patterns appear?
- Does the model forget the initial context?
- Does a longer prompt improve coherence?

Extension (+20 min) — `02_extension.py`

Build a **TinyAttnLM**: same data, same training loop, but replace the LSTM with a single self-attention layer + causal mask. Compare qualitatively with the LSTM outputs.

`labs/lab_s1/`

The Fundamental Idea of Self-Attention

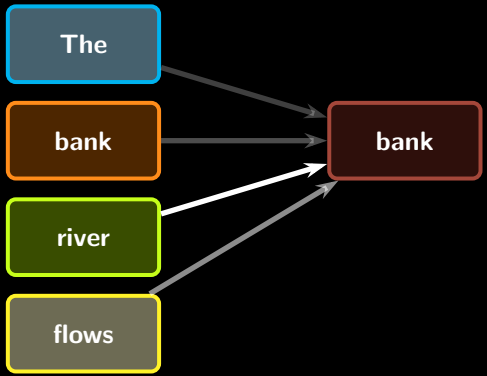
Moderate analogy

Think of self-attention as an **internal search engine**. For each word, the model asks:

- What am I looking for right now? **Query**
- What does every other word offer? **Key**
- What information can each word contribute? **Value**

Why it matters

Every token can directly access all other tokens, instead of waiting for information to travel step by step through a recurrent chain.



The Fundamental Idea of Self-Attention

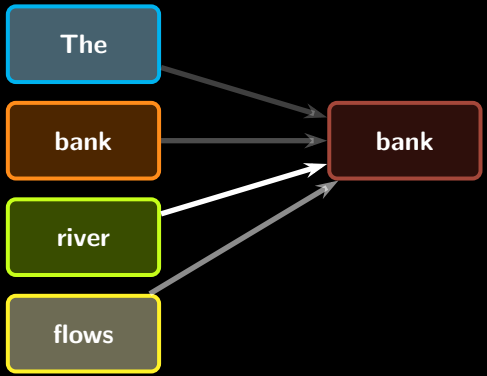
Moderate analogy

Think of self-attention as an **internal search engine**. For each word, the model asks:

- What am I looking for right now? **Query**
- What does every other word offer? **Key**
- What information can each word contribute? **Value**

Why it matters

Every token can directly access all other tokens, instead of waiting for information to travel step by step through a recurrent chain.



The Fundamental Idea of Self-Attention

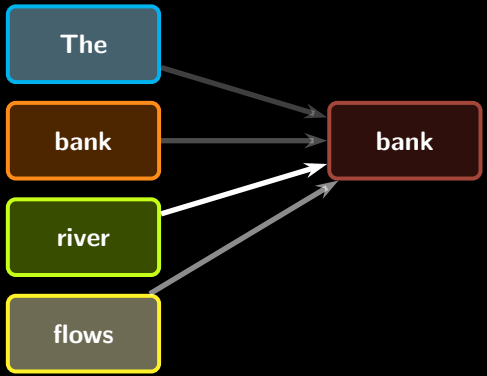
Moderate analogy

Think of self-attention as an **internal search engine**. For each word, the model asks:

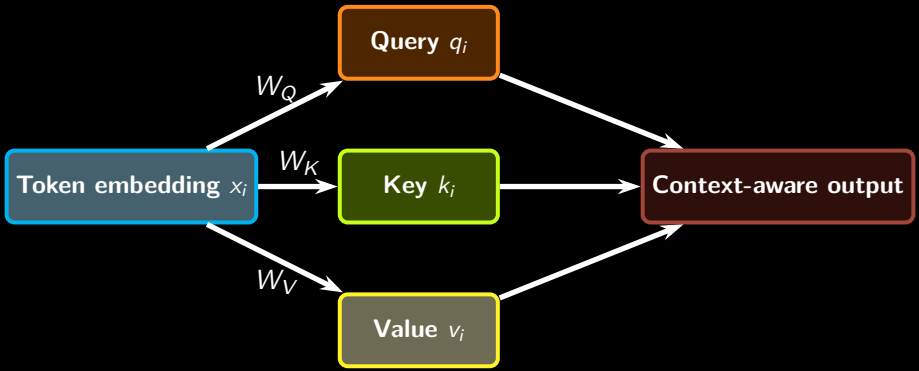
- What am I looking for right now? **Query**
- What does every other word offer? **Key**
- What information can each word contribute? **Value**

Why it matters

Every token can directly access all other tokens, instead of waiting for information to travel step by step through a recurrent chain.



Queries, Keys, Values in One Diagram



Plain-language view

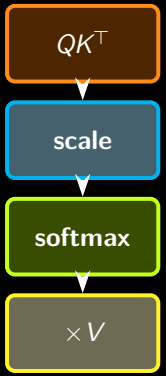
The model projects the same token representation into three roles because **matching** and **content transfer** are not the same operation.

Scaled Dot-Product Attention

Optional math lens

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top$: how much each token matches the others
- division by $\sqrt{d_k}$: keeps scores numerically stable
- softmax: turns scores into attention weights
- multiplication by V: mixes the content accordingly

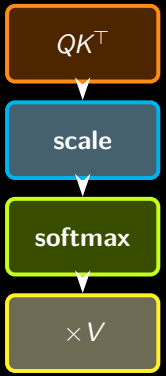


Scaled Dot-Product Attention

Optional math lens

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top$: how much each token matches the others
- division by $\sqrt{d_k}$: keeps scores numerically stable
- softmax: turns scores into attention weights
- multiplication by V: mixes the content accordingly

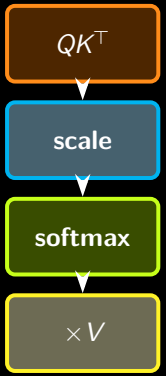


Scaled Dot-Product Attention

Optional math lens

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top$: how much each token matches the others
- division by $\sqrt{d_k}$: keeps scores numerically stable
- softmax: turns scores into attention weights
- multiplication by V: mixes the content accordingly

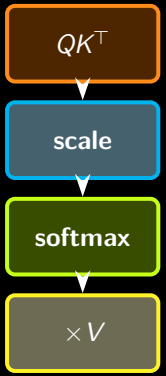


Scaled Dot-Product Attention

Optional math lens

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top$: how much each token matches the others
- division by $\sqrt{d_k}$: keeps scores numerically stable
- softmax: turns scores into attention weights
- multiplication by V: mixes the content accordingly

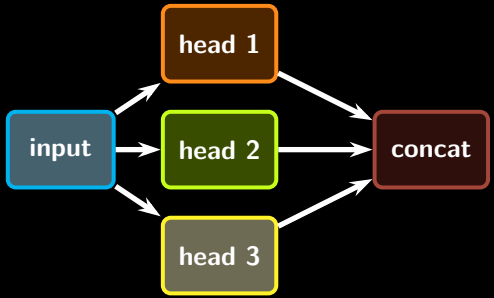


Why Multi-Head Attention Helps

Useful analogy

A single head is one lens. Multi-head attention is a **panel of lenses** looking at the same sentence from different representational angles.

- one head may focus on subject-verb agreement
- another may track entities or references
- another may emphasize local syntax

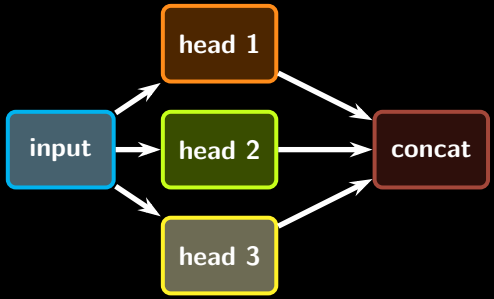


$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W_O$$

Why Multi-Head Attention Helps

Useful analogy

A single head is one lens. Multi-head attention is a **panel of lenses** looking at the same sentence from different representational angles.



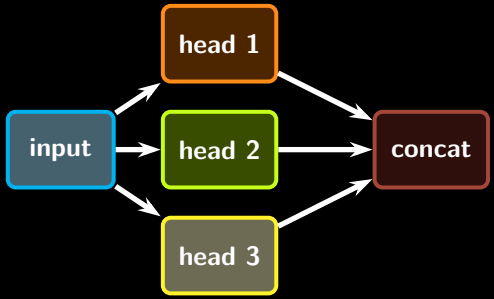
- one head may focus on subject-verb agreement
- another may track entities or references
- another may emphasize local syntax

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W_O$$

Why Multi-Head Attention Helps

Useful analogy

A single head is one lens. Multi-head attention is a **panel of lenses** looking at the same sentence from different representational angles.



- one head may focus on subject-verb agreement
- another may track entities or references
- another may emphasize local syntax

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W_O$$

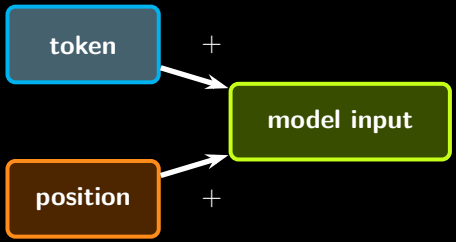
Positional Encoding: Injecting Order

The issue

Self-attention alone does not know whether a word comes first, last, or in the middle.

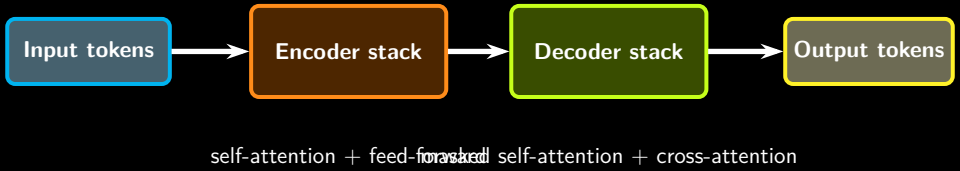
The fix

Add a positional signal to the token embeddings so that the model can reason about order.



$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

High-Level Transformer Architecture



Reading tip

Most modern diagrams are variations of this pattern. Once you recognize the blocks, you can orient yourself quickly.

Three Main Transformer Families

Family	Example	Best known use
Encoder-only	BERT	classification, NER, sentence understanding
Decoder-only	GPT	autoregressive text generation
Encoder-decoder	T5	translation, summarization, text-to-text tasks

Quick heuristic

If the task is mainly **understand**, think encoder. If the task is mainly **generate**, think decoder. If it is **map one sequence to another**, think encoder-decoder.

Why Transformers Changed Everything

Compared with RNNs

- more parallelism during training
- easier access to long-range context
- better scaling with data and compute

Practical consequence

Once scaling worked, Transformers became the backbone of modern LLMs, multimodal models, and many reasoning systems.

Why Transformers Changed Everything

Compared with RNNs

- more parallelism during training
- easier access to long-range context
- better scaling with data and compute

Practical consequence

Once scaling worked, Transformers became the backbone of modern LLMs, multimodal models, and many reasoning systems.

Why Transformers Changed Everything

Compared with RNNs

- more parallelism during training
- easier access to long-range context
- better scaling with data and compute

Practical consequence

Once scaling worked, Transformers became the backbone of modern LLMs, multimodal models, and many reasoning systems.

Manual Attention Exercise

Advanced extension

Given a 3-token sequence and matrices Q, K, and V, compute:

$$QK^\top \quad \rightarrow \quad \frac{QK^\top}{\sqrt{d_k}} \quad \rightarrow \quad \text{softmax} \quad \rightarrow \quad \text{Attention}(Q, K, V)$$

What this develops

- matrix multiplication intuition
- row-wise normalization
- how weights become contextual mixing

Use in class:
Best for faster groups or as a guided whiteboard activity after the main explanation.

Manual Attention Exercise

Advanced extension

Given a 3-token sequence and matrices Q, K, and V, compute:

$$QK^\top \quad \rightarrow \quad \frac{QK^\top}{\sqrt{d_k}} \quad \rightarrow \quad \text{softmax} \quad \rightarrow \quad \text{Attention}(Q, K, V)$$

What this develops

- matrix multiplication intuition
- row-wise normalization
- how weights become contextual mixing

Use in class:
Best for faster groups or as a guided whiteboard activity after the main explanation.

Manual Attention Exercise

Advanced extension

Given a 3-token sequence and matrices Q, K, and V, compute:

$$QK^\top \quad \rightarrow \quad \frac{QK^\top}{\sqrt{d_k}} \quad \rightarrow \quad \text{softmax} \quad \rightarrow \quad \text{Attention}(Q, K, V)$$

What this develops

- matrix multiplication intuition
- row-wise normalization
- how weights become contextual mixing

Use in class:
Best for faster groups or as a guided whiteboard activity after the main explanation.

Session 2 Lab

Baseline task

Load a small pre-trained model via Hugging Face, inspect attention weights on one sentence, and observe that different heads capture different relations.

Suggested observations

- local vs long-range focus
- punctuation and structural tokens
- entity-to-entity or subject-to-verb links

Extension:
Manually compute attention on a tiny example and compare with the library output.

Session 2 Lab

Baseline task

Load a small pre-trained model via Hugging Face, inspect attention weights on one sentence, and observe that different heads capture different relations.

Suggested observations

- local vs long-range focus
- punctuation and structural tokens
- entity-to-entity or subject-to-verb links

Extension:
Manually compute attention on a tiny example and compare with the library output.

Session 2 Lab

Baseline task

Load a small pre-trained model via Hugging Face, inspect attention weights on one sentence, and observe that different heads capture different relations.

Suggested observations

- local vs long-range focus
- punctuation and structural tokens
- entity-to-entity or subject-to-verb links

Extension:
Manually compute attention on a tiny example and compare with the library output.

What Students Should Retain

1. Neural architectures evolved by adding useful assumptions about data structure.
2. CNNs encode locality, RNNs encode sequentiality, attention enables direct access to context.
3. LSTMs improved memory, but attention removed the recurrent bottleneck more radically.
4. Transformers combine self-attention, positional information, and feed-forward blocks into a scalable architecture.
5. GPT, BERT, and T5 are not mysterious new species; they are architectural variants of the same family.

What Students Should Retain

1. Neural architectures evolved by adding useful assumptions about data structure.
2. CNNs encode locality, RNNs encode sequentiality, attention enables direct access to context.
3. LSTMs improved memory, but attention removed the recurrent bottleneck more radically.
4. Transformers combine self-attention, positional information, and feed-forward blocks into a scalable architecture.
5. GPT, BERT, and T5 are not mysterious new species; they are architectural variants of the same family.

What Students Should Retain

1. Neural architectures evolved by adding useful assumptions about data structure.
2. CNNs encode locality, RNNs encode sequentiality, attention enables direct access to context.
3. LSTMs improved memory, but attention removed the recurrent bottleneck more radically.
4. Transformers combine self-attention, positional information, and feed-forward blocks into a scalable architecture.
5. GPT, BERT, and T5 are not mysterious new species; they are architectural variants of the same family.

What Students Should Retain

1. Neural architectures evolved by adding useful assumptions about data structure.
2. CNNs encode locality, RNNs encode sequentiality, attention enables direct access to context.
3. LSTMs improved memory, but attention removed the recurrent bottleneck more radically.
4. Transformers combine self-attention, positional information, and feed-forward blocks into a scalable architecture.
5. GPT, BERT, and T5 are not mysterious new species; they are architectural variants of the same family.

What Students Should Retain

1. Neural architectures evolved by adding useful assumptions about data structure.
2. CNNs encode locality, RNNs encode sequentiality, attention enables direct access to context.
3. LSTMs improved memory, but attention removed the recurrent bottleneck more radically.
4. Transformers combine self-attention, positional information, and feed-forward blocks into a scalable architecture.
5. GPT, BERT, and T5 are not mysterious new species; they are architectural variants of the same family.

Suggested Resources

Core

- Jay Alammar, *The Illustrated Transformer*
- Harvard NLP, annotated *Attention Is All You Need*

Before the next module

- review tokenization intuition
- explore one Hugging Face model card

Thank you!

Questions & Discussion

Dr. Papa-Séga WADE

papa-sega.wade@m4x.org /LinkedIn/papa sega wade

"From neurons to Transformers, the narrative should now feel continuous."